SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1441 COURSE NAME: Compiler Design for Engineers

---

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: **Total Marks:50**

Mode: **Take-Home** **Hours : 1 Day**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I _K.Madhulika(192524190)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Madhulika

Q. No	Error Identification (20)	Root Cause Analysis (30)	Correction & Justification (25)	Application of Concepts (15)	Clarity & Presentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 30**	**/ 25**	**/ 15**	**/ 10**	**/ 100**

Annexure A

ERROR ANALYSIS TASK

Q1. Full Compiler Phase Error Analysis (BTL4) (10 Marks)

The following C program is intended to perform arithmetic operations, conditional checks, and looping constructs. However, it contains multiple errors across different phases of compilation including lexical, syntax, semantic, logical, and runtime errors.

```
#include <stdio.h>

int main() {
int num1 = 25;
float x = 3.5
int y;

y = num1 + 10

if(y == 35 {
printf("Correct Value\n")
}

int arr[3] = {1,2,3};
printf("%d\n", arr[3]);

int *p;
*p = 20;

int z = 10 / 0;

while(num1 < 30) {
printf("%d\n", num1);
num1++;
}

return 0;
}
```

Tasks:

(a) Identify all errors with description
(b) Classify errors into lexical, syntax, semantic, logical, and runtime errors
(c) Explain which errors are detected during compilation and which occur at runtime

(d) Rewrite the corrected version of the program
(e) Analyze the impact of division by zero and pointer misuse
(f) Suggest improvements in compiler design for better error detection

Q2. Advanced Lexical Error Identification (BTL4) (10 Marks)

The following C program contains multiple lexical errors due to invalid identifiers, malformed constants, and incorrect symbols. Carefully analyze the program and answer the questions.

```
#include <stdio.h>

int main() {
int 5num = 10;
float value& = 2.5;
char ch = 'a';
int total = 10#20;
int _data = 15;
int data@1 = 25;
printf("Sample Program);
int y = 0xH12;
float f = 3.4.5;
char str[] = "Hello;
return 0;
}
```

Tasks:

(a) Identify all lexical errors with invalid tokens
(b) Explain which lexical rule is violated (identifier, constant, literal, symbol)
(c) Rewrite each invalid token into a valid token
(d) Explain how a lexical analyzer processes such errors
(e) Design a DFA rule to detect invalid identifiers starting with digits
(f) Suggest improvements in lexical error reporting

Q3. Intermediate Code and Optimization Analysis (BTL4) (10 Marks)

The following C program demonstrates arithmetic computation and optimization concepts but contains inefficiencies and errors.

```
#include <stdio.h>

int main() {
int a = 5;
int b = 10;
int c;

c = a * b + a * b + b * a;

if(a = 5) {
printf("Check\n");
}

int arr[2] = {1,2};
printf("%d\n", arr[2]);

int *ptr;
*ptr = 10;

return 0;
}
```

Tasks:

(a) Identify all errors and inefficiencies in the program
(b) Classify errors according to compiler phases
(c) Generate three-address code (TAC) for the expression
(d) Apply DAG representation to eliminate redundant computations
(e) Generate optimized target code with minimal register usage
(f) Analyze the runtime impact of array bounds violation and pointer misuse

Q4. Error Identification and Classification (BTL4) (10 Marks)

Analyze the following C program.

```
#include <stdio.h>

int main() {
   int x = 10
   int y = 20;
   int z = x + y;
   printf("%d\n", z);
   return 0;
}
```

Tasks:

(a) Identify all errors in the program.
(b) Classify each error as lexical, syntax, semantic, logical, or runtime.
(c) Rewrite the corrected program.

Q5. Compiler Error Analysis (BTL4) (10 Marks)

Analyze the following C program.

```
#include <stdio.h>

int main() {
   int arr[2] = {5,10};
   printf("%d\n", arr[3]);

   int *ptr;
   *ptr = 50;

   return 0;
}
```

Tasks:

(a) Identify the runtime errors in the program.
(b) Explain why these errors occur.
(c) Rewrite the corrected version of the program.

Bottom of Form

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors

Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

CO1 AT2

float x = 3.5

syntax error because every declaration must end with a semicolon.

y = num1 + 10

syntax error. Missing semicolon.

if (y == 35 {

syntax error.

printf ("Correct value\n")

syntax error. Missing semicolon.

printf ("%d \n", arr[3]);

Semantic / Runtime error because valid indices are 0, 1, 2.

int *p; *p = 20;

Runtime error. Pointer is not pointing to valid memory.

int z = 10/0;

Runtime error. Division by zero.

while (num1 < 30)

Loop executes correctly from 25 to 29.

Lexical errors:

None present

Syntax errors:

Missing ; after float x = 3-5

Missing ; after y=num1+10

Missing) in if(y==35)

Missing ; after printf("correctValue\n")

Semantic errors:

Accessing arr[3]

Dereferencing an unintialized pointer

*p = 20

Logical errors:

None present.

Runtime errors:

Dereferencing unintialized pointer

Division by zero

Array out of bounds may also cause runtime errors.

Corrected program:

```
#include<stdio.h>
int main()
{
  int num1 = 25;
  float x = 3.5;
  int y;
  y = num1 + 10;
  if (y == 35)
  {
    printf("correct value\n");
  }
  int arr[3] = {1, 2, 3};
  printf("%d\n", arr[2]);
  int value = 20;
  int *p = &value;
  printf("%d\n", *p);
  int z = 10/2;
  printf("%d\n", z);
  while (num1 < 30)
  {
    printf("%d\n", num1);
    num1++;
  }
  return 0;
}
```

Division by zero:

→ Causes undefined behaviour.
→ Program may terminate abnormally.
→ Can generate a floating-point exception.
→ Produces incorrect program execution.

Pointer misuse:

→ May cause segmentation fault.
→ Can corrupt memory.
→ Leads to unpredictable output.
→ Makes debugging difficult.

→ ~~Lexical phase~~

1. Detect array index out-of-bounds during compilation whenever possible.
2. Warn about uninitialized pointers before execution.
3. Detect constant division by zero (10/0) during compilation.
4. Provide more descriptive error messages with line numbers.

5. Suggest automatic fixes for common syntax errors.

6. Perform stronger static analysis to identify memory-related issues.

2. (a)

int 5num = 10;

Identifier cannot start with a digit.

float value& = 2.5;

& is not allowed in an identifier.

int total = 10# 20;

Invalid symbol in an arithmetic expression.

int data@1 = 25;

@ is not allowed in identifiers.

int y = 0xH12;

Invalid hexadecimal constant.

float f = 3.4.5;

Multiple decimal points.

(b)

5num - Identifier must begin with a letter or underscore(_).

value& - Identifier cannot contain special characters like &.

10#20 - # is not a valid arithmetic operator.

data@1 - Identifier cannot contain @.

0XH12 - Hexadecimal digits can only be 0-9 and A-F (or a-f).

3.4.5 - A floating point constant can contain only one decimal point.

(c)

Invalid	correct
5num	num5
value&	value
10#20	10+20
data@1	data1
0XH12	0XA12
3.4.5	3.45

(d)

1. Reads the source code character by character.
2. Groups characters into tokens.
3. Compares each token with the lexical rules of the language.
4. If an invalid token is found, it reports a lexical error with the line number.

(e) (start)
|
Digit (0-9)
|
Letter / Digit / _
|
Reject state

→ If the first character is a digit, the FDA moves directly to the Reject State because identifiers cannot start with digits.

→ If the first character is a letter or underscore, it moves to the Accept State, allowing letters, digits, and underscores afterward.

(f)

1. Display the exact line number where the error occurs.
2. Highlight the invalid token in the source code.
3. Provide a meaningful error message.
4. Suggest a possible correction.
5. Continue scanning after detecting an error instead of stopping immediately.

3. (a) c = a * b + a * b + b * a;

a * b is calculated 3 times.

if (a = 5)

= assigns 5 to a, should use ==.

printf("%d \n", arr[2]);

arr[2] is invalid because valid indices are 0 and 1.

int *ptr; *ptr = 10;

Writing through an unitialized pointer causes a runtime error.

(b)

compiler phase	error
Lexical analysis	No lexical errors.
Syntax analysis	No syntax errors.
Semantic analysis	Assignment in if(a=5 array index issue, pointer misuse.
Optimization phase	Repeated computation
Runtime	Array out of bounds, uninitialized pointer

(c) $t_1 = a * b$

$t_2 = a * b$

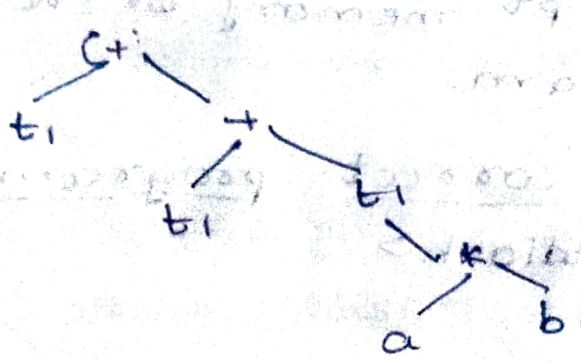
$t_3 = a * b$

$t_4 = t_1 + t_2$

$t_5 = t_4 + t_3$

$c = t_5$

(d)



optimized expression: $t_1 = a * b$

$t_2 = t_1 + t_1$

$c = t_2 + t_1$

(e)
```
MOV R1, a
MUL R1, b
MUL R1, 3
MOV c, R1
```

(f)

Array Bounds violation:

→ Accesses memory outside the array.

→ Produces unpredictable or garbage values.

→ May crash the program.

→ Causes undefined behaviour.

Pointer misuse :

→ Pointer is not initialized.

→ Writes to an invalid memory location.

→ Causes a segmentation is full.

→ May corrupt memory or terminate the program.

Optimized correct program :

```
#include <stdio.h>
int main()
{
    int a=5, b=10, c;
    c = 3* (a*b);
    if (a==5)
    {
        printf("check\n");
    }
    int arr[2] = {1, 2};
    printf("%d\n", arr[1]);
    int value = 10;
    int *ptr = &value;
    printf("%d\n", *ptr);
    return 0;
}
```

4. (a)

int x =10

Missing semicolon (;). syntax error.

int y = 20;

No error.

(b)

Missing semicolon after int x = 10

syntax error.

Lexical error : None

Syntax error : Missing ;

Semantic error : None

Logical error : None

Runtime error : None

(c) Corrected program

```
#include <stdio.h>
int main()
{
   int x=10;
   int y=20;
   int z = x+y;
   printf("%d\n", z);
   return 0;
}
```

output : 30

5. (a)

printf("%d\n", arr[3]);

Array index out of bounds

arr has only two elements indices 0 and 1. Accessing arr[3] is invalid.

int *ptr; *ptr=50;

Uninitialized pointer.

Pointer is not assigned a valid memo address before dereferencing.

(b)

1. Array out of bounds:

Array arr[2] contains only:

arr[0] = 5

arr[1] = 10

Accessing arr[3] reads memory outside the array.

This causes undefined behaviour, garba values, or program crashes.

2. Uninitialized pointer:

ptr doesnot point to any valid memory.

writing *ptr = 50 attempts to access an invalid memory location. This usually results in a segmentation fault.

(c) Corrected version of the program:

```
#include <stdio.h>
int main()
{
    int arr[2] = {5,10};
    printf("%d\n", arr[1]);
    int value = 50;
    int *ptr = &value;
    printf("%d\n", *ptr);
    return 0;
}
```

output:

10
50